جامعة الحسن الثاني بالدار البيضاء
كلية العلوم بن مسيك

FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

Faculté des Sciences Ben M'Sick (FSBM)

Université Hassan II de Casablanca

Rapport de Projet Micro-Services

Développement d'un Microservice de Gestion de Restaurant

Réalisé par :
Hafssa Bouam
Siham Ait Elmaalem
Hiba Amine
Wiam Armouli
Fatima Zahra Ajmilou
Hafsa ait hmid
Nourelislam Zaitouni
Abdelaziz Taoudi Ebourki
Oussama Rafik

Encadré par :
Pr. CHEMLAL Yman

Année Universitaire 2025-2026

Table des matières

1 Introduction **2**

2 Cahier des Charges **2**
2.1 Objectifs du Projet 2
2.2 Exigences Fonctionnelles 2
2.3 Contraintes Techniques 2

3 Conception et Architecture **3**
3.1 Analyse de la Structure 3
3.2 Schéma Architectural 3
3.3 Modélisation de la Ressource 4

4 Mise en Œuvre Technique **4**
4.1 Configuration de l'Application 5
4.2 Couche DAO et Persistance 5
4.3 Initialisation des Données (SQL) 5
4.4 Couche Contrôleur REST 6
4.4.1 Détails des méthodes 6
4.5 Gestion Globale des Exceptions 8
4.5.1 Traitement des erreurs spécifiques 8

5 Validation et Tests **9**
5.1 Documentation Interactive 10
5.2 Suite de Tests Postman 10
5.3 Résultats des Tests (Validation) 11

6 Gestion du Projet **13**

7 Conclusion **14**

1 Introduction

L'évolution des architectures logicielles vers le modèle microservices est devenue une norme dans l'industrie moderne du développement. Ce paradigme permet de décomposer une application complexe en un ensemble de services petits, autonomes et communicants via des protocoles légers. Cette approche facilite non seulement la maintenance et le déploiement continu, mais offre également une scalabilité granulaire indispensable aux applications industrielles.

Dans le cadre du module **Micro-Services**, notre équipe a été chargée de concevoir et réaliser un microservice autonome pour la gestion de la carte d'un restaurant. Ce service est construit sur l'écosystème *Spring Boot* et respecte les principes fondamentaux du style architectural *REST*.

Ce rapport présente les étapes clés de la réalisation de ce projet, depuis la conception architecturale jusqu'à la mise en œuvre technique. Nous détaillerons la structure du projet, l'implémentation des différentes couches logicielles, ainsi que les phases de validation et de test.

2 Cahier des Charges

2.1 Objectifs du Projet

L'objectif principal est de fournir une solution robuste pour la gestion des plats d'un restaurant. Le système doit permettre aux gestionnaires de manipuler la carte de manière intuitive tout en garantissant l'intégrité des données métiers. L'utilisation de *Spring Boot* a été imposée pour sa capacité à produire des applications "production-ready" avec un minimum de configuration manuelle.

2.2 Exigences Fonctionnelles

Le microservice doit supporter l'ensemble des opérations *CRUD* (Create, Read, Update, Delete) sur la ressource `Plat` :

- **Ajout** : Insertion de nouveaux plats via `POST` avec validation des prix.
- **Consultation** : Récupération globale ou spécifique par identifiant unique (`ID`).
- **Mise à jour** : Modification des propriétés d'un plat existant via `PUT`.
- **Suppression** : Retrait définitif d'un plat via `DELETE`.
- **Recherche Métier** : Un endpoint spécifique pour filtrer les plats "Healthy" (moins de 500 calories).

2.3 Contraintes Techniques

- **Persistence** : Utilisation de *Spring Data JPA* avec une base de données *H2* en mémoire pour la rapidité des tests.
- **Architecture** : Respect strict du modèle *MVC* et injection de dépendances par constructeur.
- **Sécurité** : Masquage du coût des ingrédients dans les réponses API pour des raisons de confidentialité métier.
- **Documentation** : Interface *Swagger* interactive pour faciliter l'intégration par des tiers.

3 Conception et Architecture

3.1 Analyse de la Structure

Le projet suit une structure modulaire standardisée. Cette séparation des préoccupations (*Separation of Concerns*) permet à chaque membre de l'équipe de travailler sur une couche spécifique sans interférer avec les autres.

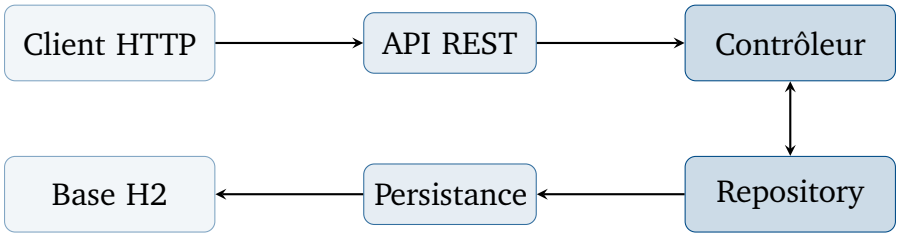
Arborescence du Projet

```
restaurant/
├─ src/
│  └─ main/
│     ├─ java/
│     │  └─ com.restaurant.restaurant/
│     │     ├─ RestaurantApplication.java
│     │     ├─ dao/
│     │     │  └─ PlatRepository.java
│     │     ├─ model/
│     │     │  └─ Plat.java
│     │     ├─ web/
│     │     │  ├─ controller/
│     │     │  │  └─ PlatController.java
│     │     │  └─ exceptions/
│     │     │     ├─ GlobalExceptionHandler.java
│     │     │     └─ ResourceNotFoundException.java
│     ├─ resources/
│     │  ├─ application.properties
│     │  ├─ schema.sql
│     │  └─ data.sql
├─ pom.xml
```

3.2 Schéma Architectural

L'architecture du microservice suit une approche modulaire permettant de découpler la logique métier des interfaces techniques (API) et de la persistance.

Flux de Données et Composants



3.3 Modélisation de la Ressource

L'entité `Plat` est le cœur de notre domaine. Elle utilise les annotations *JPA* pour le mapping relationnel et *Jackson* pour le contrôle de la sérialisation `JSON`.

Plat.java

```java
@Entity
public class Plat {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Le nom du plat est obligatoire")
    private String nom;

    private int calories;

    @Positive(message = "Le prix doit être positif")
    private double prix;

    @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
    private double coutIngredients;

    public Plat() {}

    public Plat(String nom, int calories, double prix, double
    ↪ coutIngredients) {
        this.nom = nom;
        this.calories = calories;
        this.prix = prix;
        this.coutIngredients = coutIngredients;
    }
}
```

4 Mise en Œuvre Technique

4.1 Configuration de l'Application

Le fichier `application.properties` définit le comportement de la base de données *H2* et du moteur *Hibernate*. Nous avons activé le mode `defer-datasource-initialization` pour garantir que les scripts `SQL` s'exécutent après la création des tables par *JPA*.

application.properties

```
spring.datasource.url=jdbc:h2:mem:restaurantdb
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

spring.jpa.hibernate.ddl-auto=none
spring.jpa.defer-datasource-initialization=true
spring.sql.init.mode=always
spring.jpa.show-sql=true
```

4.2 Couche DAO et Persistance

Le repository utilise l'interface `JpaRepository` qui fournit par défaut toutes les méthodes *CRUD*. Nous y avons ajouté une méthode de requête dérivée (*Query Method*) pour le filtrage par calories.

PlatRepository.java

```java
@Repository
public interface PlatRepository extends JpaRepository<Plat, Long> {
    List<Plat> findByCaloriesLessThan(int maxCalories);
}
```

4.3 Initialisation des Données (SQL)

Pour assurer la portabilité et faciliter les tests, nous utilisons deux fichiers `SQL` chargés automatiquement au démarrage.

schema.sql

```sql
CREATE TABLE plat (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    calories INT NOT NULL,
    prix DOUBLE NOT NULL,
    cout_ingredients DOUBLE NOT NULL
);
```

data.sql

```sql
INSERT INTO plat (nom, calories, prix, cout_ingredients)
VALUES ('Salade César', 350, 12.50, 4.20);
INSERT INTO plat (nom, calories, prix, cout_ingredients)
VALUES ('Burger Gourmet', 850, 18.00, 7.50);
INSERT INTO plat (nom, calories, prix, cout_ingredients)
VALUES ('Soupe Potiron', 150, 8.00, 2.10);
INSERT INTO plat (nom, calories, prix, cout_ingredients)
VALUES ('Pizza 4 Fromages', 1100, 15.50, 5.80);
INSERT INTO plat (nom, calories, prix, cout_ingredients)
VALUES ('Poke Bowl Saumon', 450, 14.00, 6.00);
```

4.4 Couche Contrôleur REST

La classe `PlatController` orchestre les requêtes entrantes. Elle délègue la persistance au repository et assure la transformation des données en réponses HTTP.

Signature de PlatController.java

```java
@RestController
@RequestMapping("/plats")
public class PlatController {
    private final PlatRepository platRepository;

    public PlatController(PlatRepository platRepository);
    public List<Plat> listePlats();
    public Plat getPlatById(Long id);
    public List<Plat> getPlatsHealthy();
    public ResponseEntity<Plat> ajouterPlat(Plat plat);
    public ResponseEntity<Plat> modifierPlat(Long id, Plat platDetails);
    public ResponseEntity<Void> deletePlat(Long id);
}
```

4.4.1 Détails des méthodes

Injection de dépendances

```java
public PlatController(PlatRepository platRepository) {
    this.platRepository = platRepository;
}
```

Le constructeur garantit que le contrôleur dispose toujours d'une instance valide du repository pour interagir avec la base de données.

Récupération globale

```java
@GetMapping
```

```
public List<Plat> listePlats() {
    return platRepository.findAll();
}
```

Cette méthode récupère l'intégralité des plats via la méthode standard `findAll()` de Spring Data JPA.

Récupération par identifiant

```
@GetMapping("/{id}")
public Plat getPlatById(@PathVariable Long id) {
    return platRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Plat non trouvé
            ↪  avec id : " + id));
}
```

Permet de consulter un plat spécifique. En cas d'absence, elle lève une exception `ResourceNotFoundExcep`

Filtre métier Healthy

```
@GetMapping("/healthy")
public List<Plat> getPlatsHealthy() {
    return platRepository.findByCaloriesLessThan(500);
}
```

Implémente le filtrage des plats considérés comme sains (moins de 500 calories).

Création de ressource

```
@PostMapping
public ResponseEntity<Plat> ajouterPlat(@Valid @RequestBody Plat plat) {
    Plat platAjoute = platRepository.save(plat);
    return new ResponseEntity<>(platAjoute, HttpStatus.CREATED);
}
```

Gère l'ajout d'un plat avec validation des données et retourne un code `201 Created`.

Mise à jour complète

```
@PutMapping("/{id}")
public ResponseEntity<Plat> modifierPlat(@PathVariable Long id, @Valid
↪  @RequestBody Plat platDetails) {
    Plat plat = platRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Plat non trouvé
            ↪  avec id : " + id));

    plat.setNom(platDetails.getNom());
    plat.setCalories(platDetails.getCalories());
    plat.setPrix(platDetails.getPrix());
```

```
    plat.setCoutIngredients(platDetails.getCoutIngredients());

    final Plat updatedPlat = platRepository.save(plat);
    return ResponseEntity.ok(updatedPlat);
}
```

Met à jour une ressource existante après avoir vérifié son existence.

Suppression de ressource

```
@DeleteMapping("/{id}")
public ResponseEntity<Void> deletePlat(@PathVariable Long id) {
    if (!platRepository.existsById(id)) {
        throw new ResourceNotFoundException("Plat non trouvé avec id : " +
        ↪  id);
    }
    platRepository.deleteById(id);
    return ResponseEntity.noContent().build();
}
```

Supprime un plat et renvoie un code `204 No Content`.

4.5 Gestion Globale des Exceptions

Le `GlobalExceptionHandler` centralise le traitement des erreurs pour garantir des réponses JSON uniformes.

Signature de GlobalExceptionHandler.java

```
@ControllerAdvice
public class GlobalExceptionHandler {
    public ResponseEntity<Object>
    ↪  handleResourceNotFoundException(ResourceNotFoundException ex);
    public ResponseEntity<Object>
    ↪  handleValidationExceptions(MethodArgumentNotValidException ex);
    public ResponseEntity<Object>
    ↪  handleNoResourceFound(NoResourceFoundException ex);
    public ResponseEntity<Object> handleGlobalException(Exception ex);
}
```

4.5.1 Traitement des erreurs spécifiques

Ressource non trouvée (404)

```
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<Object>
↪  handleResourceNotFoundException(ResourceNotFoundException ex) {
    Map<String, Object> body = new HashMap<>();
    body.put("timestamp", LocalDateTime.now());
```

```
    body.put("message", ex.getMessage());
    body.put("status", HttpStatus.NOT_FOUND.value());
    return new ResponseEntity<>(body, HttpStatus.NOT_FOUND);
}
```

Erreurs de validation (400)

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Object>
↪ handleValidationExceptions(MethodArgumentNotValidException ex) {
    Map<String, Object> errors = new HashMap<>();
    ex.getBindingResult().getFieldErrors()
            .forEach(error -> errors.put(error.getField(),
            ↪  error.getDefaultMessage()));

    Map<String, Object> body = new HashMap<>();
    body.put("timestamp", LocalDateTime.now());
    body.put("errors", errors);
    body.put("status", HttpStatus.BAD_REQUEST.value());
    return new ResponseEntity<>(body, HttpStatus.BAD_REQUEST);
}
```

URL non trouvée (404)

```
@ExceptionHandler(NoResourceFoundException.class)
public ResponseEntity<Object> handleNoResourceFound(NoResourceFoundException
↪ ex) {
    Map<String, Object> body = new HashMap<>();
    body.put("timestamp", LocalDateTime.now());
    body.put("message", "Endpoint non trouvé");
    body.put("status", HttpStatus.NOT_FOUND.value());
    return new ResponseEntity<>(body, HttpStatus.NOT_FOUND);
}
```

Sécurité par défaut (500)

```
@ExceptionHandler(Exception.class)
public ResponseEntity<Object> handleGlobalException(Exception ex) {
    Map<String, Object> body = new HashMap<>();
    body.put("timestamp", LocalDateTime.now());
    body.put("message", "Une erreur interne s'est produite.");
    body.put("status", HttpStatus.INTERNAL_SERVER_ERROR.value());
    return new ResponseEntity<>(body, HttpStatus.INTERNAL_SERVER_ERROR);
}
```

5 Validation et Tests

5.1 Documentation Interactive

Grâce à *SpringDoc OpenAPI*, notre API s'auto-documente. En accédant à `/swagger-ui/index.html`, les développeurs peuvent visualiser les schémas de données et tester les requêtes sans outils tiers.

5.2 Suite de Tests Postman

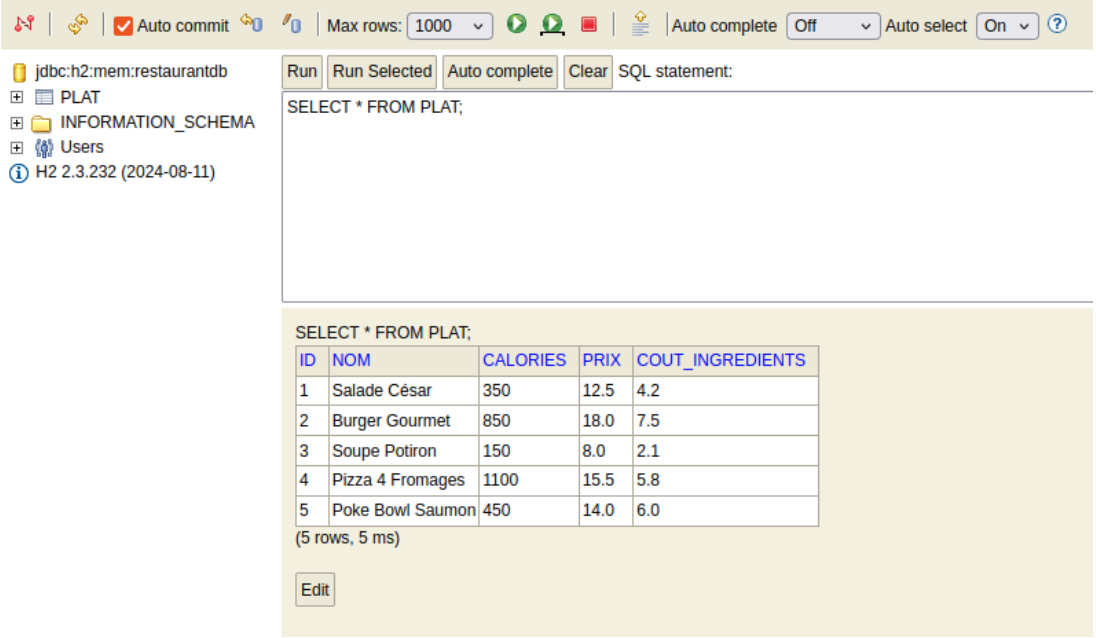
Une collection *Postman* a été créée pour automatiser la validation des exigences. Chaque requête inclut des scripts de test en *JavaScript* pour vérifier les codes de retour et la structure du `JSON`.

Collection Postman (Extraits)

```
{
  "name": "Ajouter Plat",
  "request": {
    "method": "POST",
    "url": "http://localhost:8080/plats",
    "body": { "mode": "raw", "raw": "{\"nom\": \"Salade César\",
    ↪  \"calories\": 450, \"prix\": 12.0}" }
  },
  "event": [ {
      "listen": "test",
      "script": { "exec": [
          "pm.test('Status est 201', function () {
          ↪  pm.response.to.have.status(201); });",
          "pm.test('Filtrage JSON : coutIngredients absent', function ()
          ↪  { ",
          "    pm.expect(pm.response.json().coutIngredients)
          .to.be.undefined; });"
      ] }
  } ]
}
```

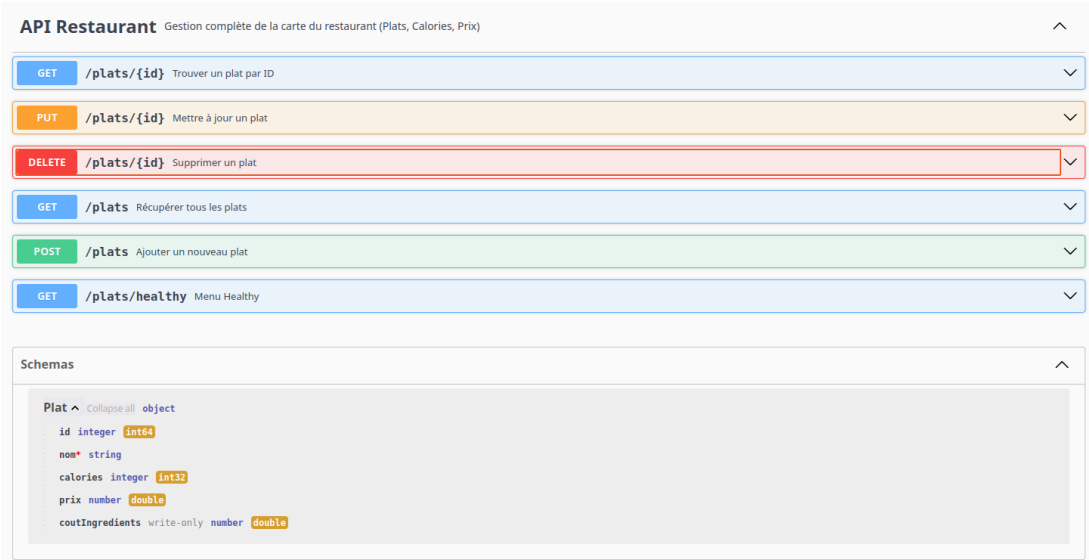
5.3 Résultats des Tests (Validation)

Console H2 : État de la Base de Données



La console H2 permet de visualiser en temps réel l'état de la base de données relationnelle en mémoire. Comme illustré ci-dessus, les tables sont correctement générées par Hibernate, et les données initiales définies dans le fichier `data.sql` sont bien présentes, confirmant le bon fonctionnement de la couche de persistance dès le démarrage.

Documentation API (Swagger UI)



L'interface Swagger UI offre une vue exhaustive de tous les points d'accès (endpoints) exposés par le microservice. Elle permet non seulement de consulter les schémas de données attendus pour chaque requête, mais aussi d'exécuter des tests interactifs directement depuis le navigateur, facilitant ainsi grandement le travail d'intégration pour les clients de l'API.

Test : Ajout d'un Plat (Succès 201)

Requête

```http
POST /plats HTTP/1.1
Content-Type: application/json

{
  "nom": "Pasta Pesto",
  "calories": 450,
  "prix": 14.5,
  "coutIngredients": 4.0
}
```

Réponse

```http
HTTP/1.1 201 Created

{
  "id": 7,
  "nom": "Pasta Pesto",
  "calories": 450,
  "prix": 14.5
}
```

Cette interaction démontre la création d'une ressource. On observe que l'API retourne un code `201 Created`. Conformément aux exigences, le champ `coutIngredients` envoyé dans la requête est bien masqué dans la réponse JSON, protégeant les données de coût.

Test : Recherche Métier Healthy (Calories < 500)

Requête

```http
GET /plats/healthy HTTP/1.1
```

Réponse

```http
HTTP/1.1 200 OK

[
  { "id": 1, "nom": "Salade César", "calories": 350, "prix": 12.5 },
  { "id": 3, "nom": "Soupe Potiron", "calories": 150, "prix": 8.0 },
  { "id": 5, "nom": "Poke Bowl Saumon", "calories": 450, "prix": 14.0 },
  { "id": 7, "nom": "Pasta Pesto", "calories": 450, "prix": 14.5 }
]
```

L'appel au endpoint dédié au menu "Healthy" confirme l'efficacité du filtrage. Le serveur renvoie exclusivement les plats dont l'apport calorique est inférieur au seuil de 500 calories, validant l'implémentation de la requête dérivée dans la couche DAO.

Test : Validation des Données (Erreur 400)

Requête

```
POST /plats HTTP/1.1
```

```
{
  "nom": "",
  "prix": -10
}
```

Réponse

```
HTTP/1.1 400 Bad Request
```

```
{
  "status": 400,
  "errors": {
    "prix": "Le prix doit être positif",
    "nom": "Le nom du plat est obligatoire"
  },
  "timestamp": "2026-03-26T16:55:53.790"
}
```

L'envoi de données invalides déclenche immédiatement une erreur `400 Bad Request`. Le dictionnaire d'erreurs renvoyé par le `GlobalExceptionHandler` permet au client de corriger précisément les champs posant problème (nom vide et prix négatif).

Test : Ressource non trouvée (Erreur 404)

Requête

```
GET /plats/999 HTTP/1.1
```

Réponse

```
HTTP/1.1 404 Not Found
```

```
{
  "status": 404,
  "message": "Plat non trouvé avec id : 999",
  "timestamp": "2026-03-26T16:56:33.915"
}
```

La gestion des identifiants inexistants est illustrée ici. Le service répond par un code `404 Not Found` standard, accompagné d'un message explicatif, assurant une gestion robuste des exceptions métier à travers toute l'application.

6 Gestion du Projet

La répartition des tâches a permis une progression quasi parallèle :

- **Architecture** : Configuration initiale, *Maven*, dépendances et structure.
- **Conception** : Modèle de données et règles de validation.

— **Data Access** : Scripts `SQL` et couche de persistance.
— **Contrôleur** : Implémentation des endpoints `GET` et `POST`.
— **Logique Métier** : Développement de la logique métier “Healthy”.
— **Mise à jour** : Implémentation de la méthode `PUT` pour les modifications.
— **Suppression** : Gestion de la suppression et des codes *No Content*.
— **Sécurité et Exceptions** : Sécurisation via le gestionnaire d’exceptions.
— **Documentation et Tests** : Documentation *Swagger* et tests *Postman*.

7 Conclusion

En conclusion, ce projet a permis de concevoir et d’implémenter un microservice de gestion de restaurant complet, respectant rigoureusement les standards industriels actuels. L’architecture adoptée, reposant sur *Spring Boot* et *Spring Data JPA*, garantit une séparation claire des responsabilités et une persistance des données efficace via une base *H2* en mémoire.

L’intégration de *Bean Validation* et d’un gestionnaire global d’exceptions a renforcé la robustesse du système, tandis que l’utilisation de *Jackson* pour le filtrage `JSON` a assuré la confidentialité des données sensibles comme le coût des ingrédients. Enfin, l’exposition d’une documentation interactive via *Swagger* et la validation par une suite de tests *Postman* rigoureuse confirment la qualité et la conformité du service aux exigences du cahier des charges.